

HireProof.tech performance & SEO review

Overview

HireProof is a job-scam verification service built with **Next.js 16** and **React 19**. The site offers a free job-post auditing interface at `/audit` and includes interactive modules like red-flag training and developer integrations. It uses a **public manifest** to make the site installable (PWA) and centralizes SEO metadata in `lib/seo.ts` for canonical URLs and structured data `【filecite↑turn5file0↑L9-L17】`. The repository indicates the stack uses Tailwind CSS 4, Framer Motion and integrates with AI providers via the Vercel AI SDK `【filecite↑turn2file1↑L269-L274】`. This review covers current performance indicators and provides actionable steps to improve **Core Web Vitals** and SEO.

Current performance observations

Metric / feature	Evidence	Assessment
Time to First Byte (TTFB)	A <code>curl</code> benchmark against <code>https://hireproof.tech</code> showed a DNS lookup ~3 ms, connection ~4 ms, and a start-transfer time ≈ 74 ms .	TTFB is good; the site likely uses Next.js server-side rendering or static generation. Keep caching and global CDN distribution to maintain sub-100 ms TTFB across regions.
Largest Contentful Paint (LCP)	The landing page hero features large headings and dynamic content. There is a PWA manifest but no evidence of an optimized service worker. The LCP element is probably the hero heading/image on the audit page. Web.dev notes that LCP images should be discoverable from the HTML and preloaded if referenced via CSS ¹ .	Without explicit image preloading or <code>fetchpriority</code> , the browser may discover the hero image late. The site loads many scripts (e.g., animations, AI) that could delay style application.
Interaction to Next Paint (INP)	The site runs multiple interactive modules (claim extraction, evidence analysis) and uses heavy client-side JavaScript. Web.dev states that tasks exceeding 50 ms block the main thread and cause poor INP; breaking long tasks into smaller chunks allows the browser to respond sooner ² .	Client-side tasks such as AI inference, pattern matching and event tracking can easily create long tasks. The initial load also appears heavy due to multiple third-party scripts.

Metric / feature	Evidence	Assessment
Cumulative Layout Shift (CLS)	The README and pages show images and interactive modules inserted dynamically. Web.dev emphasises that images must include explicit width and height attributes or an aspect ratio to prevent shifts ³ .	If dynamic content (e.g., evidence results or adverts) is injected without reserved space, users may experience layout shifts. The site needs to ensure all images and asynchronous content have size placeholders.
Progressive Web App (PWA)	A <code>public/manifest.json</code> defines the app's name, icons and shortcuts <code>【filecite\turn6file0\L1-L85】</code> . However, there is no service-worker file in the repository, so caching and offline support are limited.	The manifest allows installation but the app does not leverage offline caching or background sync.
SEO and metadata	The documentation centralizes canonical URLs, Open Graph metadata and structured data in <code>lib/seo.ts</code> <code>【filecite\turn5file0\L9-L17】</code> . <code>public/manifest.json</code> includes categories and a description <code>【filecite\turn6file0\L4-L23】</code> .	Centralizing metadata is good. Additional enhancements like alt-text on critical images and consistent schema markup for job-scam detection can improve search appearance.

Issues & opportunities

- 1. LCP delays from large hero sections** – The hero section and call-to-action may contain images or backgrounds loaded via CSS. Web.dev notes that if the LCP resource is referenced only in CSS or added by JavaScript, the browser cannot discover it early ¹. Lazy-loading the hero image or loading fonts after render can delay LCP.
- 2. Heavy JavaScript and long tasks** – The site uses interactive modules to extract claims and run pattern checks. Without careful task splitting, these operations may block the main thread. Web.dev highlights that tasks longer than **50 ms** block interactions and degrade INP ². Combining multiple operations within a single function (e.g., form validation, API calls, UI updates) can create long tasks ⁴.
- 3. Potential layout shifts** – Dynamically inserted evidence blocks and images may not reserve space ahead of time. Web.dev recommends specifying `width and height attributes` or using the CSS `aspect-ratio` to reserve space ³. Without this, text may shift when results load, harming CLS.

- 4. **No service-worker caching** – While a PWA manifest exists, the absence of a custom service worker means there is no offline caching or controlled network strategy. Critical assets (icons, fonts, API responses) could be cached to improve resilience and repeat-visit performance.
- 5. **Large third-party scripts** – Integration with external APIs (Serp API, AI models) and analytics can add kilobytes of JavaScript. Each added script increases main-thread work and download time.

Recommendations

Improve Largest Contentful Paint (LCP)

- **Prioritize hero assets** – Ensure the largest hero image or heading is discoverable in the initial HTML. If the hero uses a background image in CSS, explicitly preload it via `<link rel="preload" fetchpriority="high" as="image" href="/path-to-hero.webp">` ¹. For Next.js, use the `<Image>` component with `priority` and `fetchPriority="high"` to hint that the image is the LCP element.
- **Avoid lazy loading above-the-fold images** – Do not set `loading="lazy"` on the hero image. Web.dev warns that lazy-loading LCP images delays loading and harms LCP ⁵.
- **Inline critical CSS** – Large global style sheets can block rendering even after the image is ready ⁶. Extract only the CSS required for the hero section and inline it in the document. Defer non-critical CSS via `media="print"` or dynamic imports.
- **Host fonts and images on the same origin** – LCP resources hosted on other domains require additional connections, delaying loading ⁷. Serve hero images and fonts from your own domain or use `preconnect` to external origins.

Reduce Interaction to Next Paint (INP)

- **Split up long tasks** – Break heavy functions (claim extraction, pattern analysis, analytics) into smaller tasks. For example, update UI and show spinners immediately, then defer network calls or analytics to a separate task using `setTimeout` or the `scheduler` API ⁸.
- **Leverage React concurrency features** – Use `useTransition`, `useDeferredValue` or `Suspense` to run non-critical updates without blocking user interactions. Avoid heavy computations on the client; offload analysis to API endpoints or serverless functions.
- **Reduce JavaScript bundle size** – Audit dependencies and remove unused icons, animations or libraries. Dynamically import seldom-used components (e.g., red-flag training modules, charts) so they load only when needed.

Prevent Cumulative Layout Shift (CLS)

- **Specify dimensions for all images and embeds** – Set explicit `width` and `height` attributes on `<img>` tags or define an `aspect-ratio` in CSS to reserve space ³. For responsive images using `<picture>`, set consistent aspect ratios across sources ⁹.
- **Reserve space for dynamic content** – When injecting evidence results or risk scores, wrap them in containers with predefined `min-height`. Use skeleton loaders instead of pushing content below.
- **Avoid inserting banners above existing content** – If you display notifications (e.g., BYOK alerts), insert them below the hero or reserve space ahead of time.

Enhance Progressive Web App and caching

- **Implement a service worker** – Use `next-pwa` or `workbox` to generate a service worker that precaches static assets (JS, CSS, fonts, icons) and uses the **stale-while-revalidate** strategy for API responses. Ensure caching is conservative for dynamic AI results to avoid stale data.
- **Cache fonts and icons** – The manifest lists multiple icons `【filecite\turn6file0\L21-L52】`. Precache them to avoid re-downloads on each visit. If using `next/font/local`, self-host fonts and preload them in the `<head>`.
- **Provide offline fallback** – Offer a simple offline page that explains the service and allows copying the last scanned result when network connectivity is lost.

SEO & metadata improvements

- **Structured data** – Expand the existing JSON-LD to include `Product` or `SoftwareApplication` schema that describes HireProof as an AI scam-detection tool. Provide `creator` and `offers` properties when relevant.
- **Accessible images** – Add descriptive `alt` attributes to hero images and icons. Use meaningful filenames and `title` attributes for interactive modules.
- **Optimize sitemaps and robots** – Ensure private pages (report history, settings, API routes) remain blocked from indexing, as described in the SEO docs `【filecite\turn5file0\L18-L24】`. Submit updated sitemaps to Google Search Console after deploying major changes.
- **Monitor Web Vitals in the field** – Integrate [Google's web-vitals library](#) or Vercel's Analytics to measure real-user metrics. Focus on LCP, INP and CLS thresholds to validate improvements.

Conclusion

HireProof has a solid technical foundation and already implements best practices like centralized SEO metadata and a comprehensive PWA manifest. However, the dynamic nature of the audit interface and the heavy use of client-side scripts risk degrading Core Web Vitals. By preloading and prioritizing hero assets, splitting long tasks, reserving space for dynamic content, implementing a service worker and refining metadata, the site can deliver faster, more stable pages and improved search visibility. These changes will ensure that job-seekers experience snappy, trustworthy scam detection without frustration.

1 5 6 7 Optimize Largest Contentful Paint | Articles | web.dev
<https://web.dev/articles/optimize-lcp>

2 4 8 Optimize long tasks | web.dev
<https://web.dev/articles/optimize-long-tasks>

3 9 Optimize Cumulative Layout Shift | Articles | web.dev
<https://web.dev/articles/optimize-cls>